



CSE 320 - Computer Networks

Questions

Q1. Fill in the Space

1. The Internet is often described as a **network of networks** that interconnects millions of hosts.
2. Devices such as routers and switches that forward packets are called **packet switches**.
3. In networking, a **protocol** defines the format, order, and actions taken for exchanging **messages**.
4. The computers running network applications at the edge of the Internet are called **end systems**.
5. Communication links in networks may use physical media such as fiber, copper, radio, or **sattelite**.
6. The **network edge** consists mainly of hosts such as clients and **servers**.
7. The part of the network composed of interconnected routers responsible for forwarding packets is called the **network core**.
8. In a **client-server architecture**, the server is typically an **always-on** host with a permanent IP address.
9. In **peer-to-peer architecture**, peers both request services and **provide** services to other peers.
10. A **process** communicating over the network is identified using an IP address together with a **port** number.
11. In socket communication, the socket acts as the **door between the application and the transport layer**.
12. HTTP is an example of an **application layer protocol** used by the Web.
13. Email transfer across mail servers typically uses the protocol called **SMTP**.

14. The transport layer provides logical communication between application **processes** running on different hosts.
15. The Internet transport protocol that provides reliable, ordered delivery of data is **TCP**.
16. The Internet transport protocol that provides a lightweight, connectionless service is **UDP**.
17. When sending data, the transport layer divides application messages into smaller units called **segments**.
18. The process of gathering data from multiple sockets at the sender is called **multiplexing**.
19. The process of delivering received segments to the correct socket at the receiver is called **demultiplexing**.
20. The two main functions of the network layer are **routing** and **forwarding**.

Q2. Examples of vs questions

Compare the following paradigms against each other. Provide five main differences

a. Circuit Switching vs Packet Switching

- dedicated path vs shared links
- setup phase
- efficiency
- delay characteristics
- QoS guarantees

b. Forwarding vs Routing

- local vs global decision
- data plane vs control plane
- forwarding table vs routing algorithm
- per-packet action vs path computation

c. Client-Server vs Peer-to-Peer architecture

- server existence
- scalability
- IP addressing stability
- communication pattern
- management complexity

d. Persistent HTTP vs Non-Persistent HTTP

e. Socket vs Port

f. Forwarding Table vs Routing Algorithm

Q3. Problems

a. HTTP Response Time

A client wants to download a web page from a server. The page contains:

- 1 base HTML file
- 4 referenced image objects

Assume the following:

- RTT between client and server = 80 ms
 - Transmission time for each object = 20 ms
 - Ignore DNS lookup time.
- Calculate the total time required to download the entire page using **non-persistent HTTP**.
 - Calculate the total time required using **persistent HTTP without pipelining**.

First remember the HTTP timing model:

For **non-persistent HTTP**

Response time per object = $2 \times \text{RTT} + \text{Transmission time}$

- 1 RTT $\rightarrow$ TCP connection setup
- 1 RTT $\rightarrow$ HTTP request + first byte response

Answer:

i. Non-persistent HTTP	ii. Persistent HTTP (no pipelining)
#objects: 1 HTML + 4 Images	Persistent HTTP uses one TCP connection for all objects .
Time per object: $2 \times \text{RTT} + \text{Transmission} = 2 \times 80\text{ms} + 20\text{ms} = 180\text{ms}$	Step 1. <u>Establish TCP connection</u> : $1 \times \text{RTT} = 1 \times 80\text{ms} = 80\text{ms}$
Total Time: $5 \times 180\text{ms} = 900\text{ms}$	Step 2. <u>Request HTML file</u> : $1 \times \text{RTT} + 20\text{ms} = 80\text{ms} + 20\text{ms} = 100\text{ms}$
	<u>Time so far</u> : $80\text{ms} + 100\text{ms} = 180\text{ms}$
	Now request 4 images sequentially : Each image requires:
	$1 \times \text{RTT} + 20\text{ms} = 80\text{ms} + 20\text{ms} = 100\text{ms}$
	Total for 4 images: $4 \times 100\text{ms} = 400\text{ms}$
	Total time: $180\text{ms} + 400\text{ms} = 580\text{ms}$

b. Multiplexing Example

A host is running **three applications** that send data simultaneously to the network.

Each application sends the following data:

- **Application A:** 600 bytes
- **Application B:** 1000 bytes
- **Application C:** 400 bytes

The transport layer protocol adds a **20-byte header** to every segment before passing it to the network layer.

Assume each application message is placed in **one segment**.

Questions

- How many **segments** are created by the transport layer?
- What is the **total amount of data transmitted** (including headers)?
- What percentage of the transmitted data is **header overhead**?

Answer:

- Number of applications : 3, 3 segments
- The transport layer adds a **20-byte header**.
Application A: $600+20=620$ bytes
Application B: $1000+20=1020$ bytes
Application C: $400+20=420$ bytes
- Total transmitted data: $620+1020+420 = 2060$ bytes
- Header overhead
Each segment contains a **20-byte header**: $3 \times 20 = 60$ bytes
Header overhead percentage: $60 / 2060 * 100 = 2.91 \approx \mathbf{2.9\%}$

c. Packet Transmission Delay

A host wants to send a packet of **12,000 bits** over a link with transmission rate **3 Mbps**. Assume: **Propagation delay = 8 ms**, Ignore queuing and processing delays.

Questions:

- Calculate the transmission delay.
- Calculate the total end-to-end delay over this single link.

Answer:

i. **Transmission delay: L / R**
 L = packet length in “bits”
 R = transmission rate, bit per second, bps

$L = 12000$
 $R = 3 \times 10^6 \text{ bps}$
 $d_{\text{trans}} = 12000 / (3 \times 10^6) = 4 \times 10^{-3} = 0.004 \text{ s}$

ii. **Total end-to-end delay**
 $d_{\text{total}} = d_{\text{trans}} + d_{\text{prop}} = 4\text{ms} + 8\text{ms} = 12 \text{ ms}$

Q4. Subnet Allocation

Your company received the IP block **10.40.16.0/21** from the ISP.

To verify the assignment, the network administrator performed the following tests:

- **10.40.15.200 → Responded**
- **10.40.16.10 → Responded**

This raised questions about whether the assigned block is correct.

Your company must allocate subnets for the following departments:

Department	Required Hosts
Engineering	500
Marketing	220
HR	100
IT Support	40

Answers:

10.40.16.0/21 covers

Network: 10.40.16.0

Broadcast: 10.40.23.255

So valid addresses are:

10.40.16.1 to 10.40.23.254

a) Use range calculations to determine whether 10.40.15.200 belongs to the block 10.40.16.0/21.

No: 10.40.15.200 -> is in 10.40.8.0/21 while 10.40.16.10 is in the 10.40.16.0/21 block range

b) If it does not belong to the block, determine which CIDR block most likely contains both IP addresses.

15 = 00001111

16 = 00010000

10.40.0.0/19 -> Corrected block: 10.40.0.0/19

c) Subnet the corrected block using VLSM to satisfy the host requirements of the four departments.

Fill in the table:

Department	Subnet Address	CIDR Mask
Engineering	10.40.0.0	/23
Marketing	10.40.2.0	/24
HR	10.40.3.0	/25
IT Support	10.40.3.128	/26

d) After assigning these subnets, determine the largest single contiguous CIDR block that can still be allocated from the remaining address space.

Express the answer in CIDR notation.

Used space so far:

- 10.40.0.0/23 → 10.40.0.0 to 10.40.1.255
- 10.40.2.0/24 → 10.40.2.0 to 10.40.2.255
- 10.40.3.0/25 → 10.40.3.0 to 10.40.3.127
- 10.40.3.128/26 → 10.40.3.128 to 10.40.3.191

So the large unused portion still includes: 10.40.16.0 – 10.40.31.255

That entire range is exactly:

10.40.16.0/20

Check it:

- /20 mask = 255.255.240.0
- block size in 3rd octet = 16
- covers 16–31

That is fully unused and is the **largest single aligned CIDR block** remaining.

Answer:

10.40.16.0/20

We allocated:

- 10.40.0.0/23 → covers 10.40.0.0 – 10.40.1.255
- 10.40.2.0/24 → covers 10.40.2.0 – 10.40.2.255
- 10.40.3.0/25 → covers 10.40.3.0 – 10.40.3.127
- 10.40.3.128/26 → covers 10.40.3.128 – 10.40.3.191

So the first unused address is:

- 10.40.3.192

everything from there onward is free, including 10.40.4.0 and beyond.

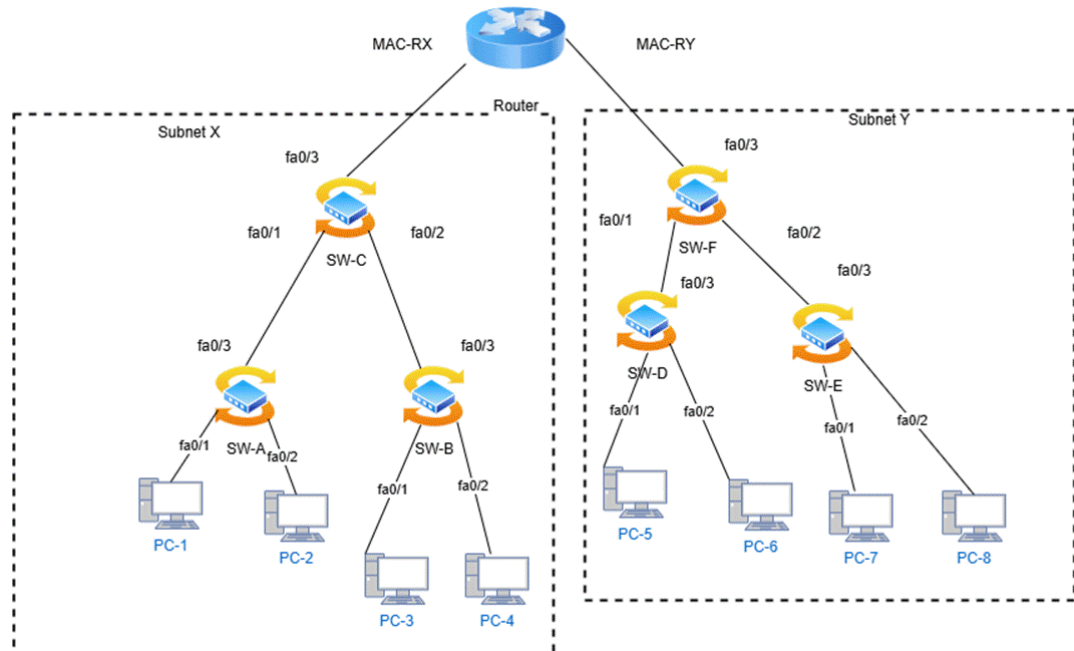
Why not just say 10.40.4.0/...?

Because CIDR blocks must be properly aligned to their size.

For example:

- a /20 block must start at a multiple of 16 in the 3rd octet:
 - 0, 16, 32, 48, ...
- so 10.40.4.0/20 is not valid as a network boundary
- 10.40.16.0/20 is valid

Q5. ARP and Switch Tables



Consider the network topology shown in the figure.

Two subnets exist:

- **Subnet X:** PC-1, PC-2, PC-3, PC-4
- **Subnet Y:** PC-5, PC-6, PC-7, PC-8

The router connects the two subnets through interfaces **MAC-RX** and **MAC-RY**.

Initial conditions at $t = 0$:

- All **ARP tables (PCs and router)** are fully up-to-date
 - All **switch MAC forwarding tables** are empty
 - All PCs know the IP address of the router
 - The router's routing table is complete
 - **No DNS lookups are required**
- a. At $t = 0.1$, PC-2 sends an IP segment to PC-5. At $t = 0.11$, the segment is successfully received by PC-5. What is the state of the **MAC forwarding tables** on all switches at $t = 0.2$?

PC-2 → SW-A → SW-C → Router → SW-F → SW-D → PC-5

1. SW-A learns MAC-PC2 → fa0/2
2. SW-C learns MAC-PC2 → fa0/1
3. SW-B learns MAC-PC2 → fa0/3 because SW-C floods unknown destination traffic

4. On Subnet Y, router sends with source **MAC-RY**
5. **SW-F** learns **MAC-RY** → **fa0/3**
6. **SW-D** learns **MAC-RY** → **fa0/3**
7. **SW-E** learns **MAC-RY** → **fa0/3**

Switch	MAC Address	Port
SW-A	MAC-PC2	fa0/2
SW-B	MAC-PC2	fa0/3
SW-C	MAC-PC2	fa0/1
SW-D	MAC-RY	fa0/3
SW-E	MAC-RY	fa0/3
SW-F	MAC-RY	fa0/3

b. At t = 0.3, PC-5 replies with an IP segment to PC-2. At t = 0.31, the segment is received by PC-2. What is the state of the MAC forwarding tables at t = 0.4?

Answer: PC-5 replies to another subnet, so it sends to **MAC-RY**.

Path: **PC-5** → **SW-D** → **SW-F** → **Router** → **SW-C** → **SW-A** → **PC-2**

1. **SW-D** learns **MAC-PC5** → **fa0/1**
2. **SW-F** learns **MAC-PC5** → **fa0/1**
3. Router now sends into Subnet X with source **MAC-RX**
4. **SW-C** learns **MAC-RX** → **fa0/3**
5. **SW-A** learns **MAC-RX** → **fa0/3**

Switch	MAC Address	Port
SW-A	MAC-PC2,MAC-RX	fa0/2, fa0/3
SW-B	MAC-PC2	fa0/3
SW-C	MAC-PC2, MAC-RX	fa0/1, fa0/3
SW-D	MAC-RY, MAC-PC5	fa0/3,fa0/1
SW-E	MAC-RY	fa0/3
SW-F	MAC-RY, MAC-PC5	fa0/3, fa0/1

c. At $t = 0.5$, PC-1 sends an IP segment to PC-8. At $t = 0.51$, the segment arrives at PC-8. What is the state of the MAC forwarding tables at $t = 0.6$?

Answer: PC-1 sends to another subnet, so destination MAC is again MAC-RX.

Path: PC-1 → SW-A → SW-C → Router → SW-F → SW-E → PC-8

1. **SW-A** learns **MAC-PC1** → fa0/1
2. **SW-C** learns **MAC-PC1** → fa0/1
3. On Subnet Y, the router sends with source **MAC-RY**, which is already known
4. **PC-8 has not transmitted before**, so switches do **not** learn MAC-PC8 yet
5. SW-F floods toward SW-D and SW-E because destination MAC-PC8 is still unknown
6. SW-E forwards to PC-8 and PC-7, but still learns nothing new from destination-only delivery

Switch	MAC Address	Port
SW-A	MAC-PC2,MAC-RX, MAC-PC1	fa0/2, fa0/3, fa0/1
SW-B	MAC-PC2	fa0/3
SW-C	MAC-PC2, MAC-RX, MAC-PC1	fa0/1, fa0/3, fa0/1
SW-D	MAC-RY, MAC-PC5	fa0/3,fa0/1
SW-E	MAC-RY	fa0/3
SW-F	MAC-RY, MAC-PC5	fa0/3, fa0/1